# Leelavathy – Medicine & Inventory Management Module

Oct 10, 2026 · @Naruti

Module 2 of 4, owned by Leelavathy: the medicine catalogue and every pharmacy's stock, with safe reserve, commit, release and restore operations.

## 1. Module description and objectives

Leelavathy's module is the single source of truth for which medicines exist and how much of each an approved pharmacy holds. It owns every stock number in the project and exposes safe `reserve`, `commit`, `release` and `restore` operations so Anusuya's request module can hold, sell and return stock without ever making it negative or double-booking it. Deepika's search module reads its data through interfaces only.

**Technology:** HTML, CSS, JavaScript, Bootstrap 5 (frontend); Java 17 with Spring Boot 3, Spring Data JPA and Hibernate, Maven (backend); MySQL 8 with Flyway migrations (database).

**Objectives**

- Maintain an admin-managed medicine catalogue with prescription flags.
- Maintain pharmacy-wise inventory: quantity, price, batch number, expiry date, low-stock threshold.
- Validate every stock change and update inventory automatically when requests are created, completed, rejected or cancelled.
- Show clear stock indicators and alerts to pharmacies.
- Store each pharmacy's contact numbers and delivery settings.
- Keep sample inventory clearly separate from genuine, pharmacy-confirmed stock.

## 2. Features

- Medicine catalogue (admin): name, generic name, strength, dosage form, category, manufacturer, `requires_prescription`.
- Pharmacy inventory: one row per pharmacy and medicine with `quantity`, `reserved_quantity`, `unit_price`, `batch_no`, `expiry_date`, `low_stock_threshold`.
- Stock status computed on read: `IN_STOCK` (green), `LOW_STOCK` (amber), `OUT_OF_STOCK` (red), `EXPIRED` (grey), plus an Expiring within 30 days flag.

- Stock operations: add medicine, stock-in, adjust with reason, confirm, soft remove, full transaction history.
- Automatic updates through `InventoryService`: reserve, commit, release, restore, all idempotent per reference code.
- Pharmacy settings: contact phone, WhatsApp number, delivery on or off, radius, fee, opening hours, accepts emergency orders. Coordinates are read-only here and edited in Divya's module.
- Data honesty: each row has `data_source` (`SAMPLE` or `LIVE`) and `last_updated_at`; the pharmacy presses Confirm to turn a sample row into a live one.
- Alerts: low stock, out of stock and expiring rows; a `LOW_STOCK` notification is sent to the pharmacy.

## 3. Workflow

1. The admin loads or edits the catalogue (demo seed: 10 medicines in 5 categories).
2. After Divya's module approves a pharmacy, this module creates a default `pharmacy_settings` row. The pharmacy edits it on `pharmacy/settings.html`.
3. The pharmacy adds a medicine from the catalogue with quantity, price, batch and expiry. Saving sets `data_source = LIVE` and writes a `STOCK_IN` transaction.
4. The pharmacy changes stock with Stock in (adds quantity) or Adjust (sets an exact count with a reason).
5. When Anusuya's module creates a request it calls `reserve`: `reserved_quantity` rises and available stock falls. On handover it calls `commit` (quantity and reserved both drop). On reject, expiry or cancel it calls `release`. If goods come back after dispatch it calls `restore`.
6. Deepika's search calls `findOffers` and `findAlternatives`, which return only approved, active, unexpired rows with enough available stock.
7. Every change writes an `inventory_transactions` row in the same database transaction.

## 4. Frontend requirements

Use HTML, CSS, JavaScript and Bootstrap 5.3. Reuse Divya's `theme.css`, `api.js`, `ui.js` and `auth-guard.js`: Navy `#102A43` primary, Teal `#0D9488` secondary, Red `#DC2626` for out-of-stock and destructive actions, backgrounds `#F1F5F9` and white, font Inter. Add only module files: `inventory.css`, `inventory.js`, `medicines.js`, `pharmacy-settings.js`. Every page needs a skeleton loader, empty state, inline validation and toasts.

| Page | Purpose and active controls |
|---|---|
| `pharmacy/inventory.html` | Summary cards (Total, In stock, Low, Out, Expiring) from `GET /inventory/summary`; search box; status filter tabs; table with columns medicine, strength, batch, quantity, reserved, available, price, expiry, status badge, source badge. Row buttons: Edit (`PUT /inventory/{id}`), Stock in (`POST .../stock-in`), Adjust (`POST .../adjust`), Confirm (`POST .../confirm`), History (`GET .../transactions`), Remove (`DELETE`). Add medicine opens `inventory-add.html`. Alerts banner from `GET /inventory/alerts`. |
| `pharmacy/inventory-add.html` | Medicine typeahead (`GET /medicines?q=`), quantity, price, batch, expiry, threshold; Save calls `POST /inventory`; Cancel returns to the list |
| `pharmacy/settings.html` | Contact phone, WhatsApp, delivery switch with radius and fee, opening hours or 24x7 switch, accepts emergency orders. Address and coordinates are read-only with a link to Divya's `profile.html`. Save calls `PUT /pharmacy-settings/me`. A preview card shows how the pharmacy appears in search. |
| `admin/medicines.html` | Tabs Medicines and Categories; Add or Edit modal; Activate or Deactivate toggle; Rx badge |
| `medicine-catalogue.html` | Public searchable card grid with Rx badge; Find pharmacies opens `search.html?medicineId=ID` (Deepika) |

UI states: red row highlight for out of stock, amber badge for low stock, amber Demo data badge for `SAMPLE` rows, green Live badge with last-updated time for `LIVE` rows, toast after every save.

## 5. Spring Boot backend requirements

- Catalogue and inventory CRUD with ownership checks; the `pharmacyId` comes from the session through `CurrentUserProvider`, never from the request body.
- Concurrency-safe stock changes: `@Lock(PESSIMISTIC_WRITE)` on the inventory row inside `@Transactional` methods, plus database `CHECK` constraints (`quantity >= 0`, `reserved_quantity >= 0`, `reserved_quantity <= quantity`).

- `StockStatusCalculator`: `EXPIRED` if expiry is today or earlier; `OUT_OF_STOCK` if available is 0; `LOW_STOCK` if available is at most the threshold; otherwise `IN_STOCK`.
- Direct pharmacy create, edit, stock-in and adjust set `data_source = LIVE`; seeded rows stay `SAMPLE`.
- Only `APPROVED` and active pharmacies can change inventory, otherwise `PHARMACY_NOT_APPROVED`.
- Implement `MedicineLookup`, `InventoryQueryService` (`findOffers`, `findAlternatives`), `InventoryService` (`reserve`, `commit`, `release`, `restore`, idempotent per inventory row, reference code and transaction type) and `PharmacySettingsLookup`.
- `findAlternatives` returns same generic name, strength and form from other medicines plus partial stock of the requested medicine, flagged `substitute`.
- Listen to `PharmacyApprovedEvent` to create default settings; create lazily if missing.
- Call `NotificationPublisher` with `LOW_STOCK` when a row crosses its threshold.
- Flyway versions `V200` to `V299`; `InventoryDemoDataLoader` (order 200) seeds 5 categories, 10 medicines and sample inventory for the demo pharmacies (including one out-of-stock, one low-stock and one near-expiry row).

```
com.emergencymed.inventory
  controller/ MedicineController, CategoryController, InventoryController,
              PharmacySettingsController, AdminInventoryController
  service/    MedicineService, CategoryService, InventoryManagementService,
              PharmacySettingsService, StockStatusCalculator
  service/impl/ MedicineLookupImpl, InventoryQueryServiceImpl,
              InventoryServiceImpl, PharmacySettingsLookupImpl
  repository/ MedicineCategoryRepository, MedicineRepository,
PharmacySettingsRepository,
              PharmacyInventoryRepository, InventoryTransactionRepository
  entity/     MedicineCategory, Medicine, PharmacySettings,
PharmacyInventory, InventoryTransaction
  dto/        request and response records, InventoryRowResponse
(stockStatus, dataSource)
  listener/   PharmacyApprovedListener
  demo/       InventoryDemoDataLoader
src/main/resources/db/migration  V200__m2_catalogue.sql,
V210__m2_pharmacy_settings_inventory.sql
src/main/resources/static/pharmacy/ inventory.html, inventory-add.html,
settings.html
src/main/resources/static/admin/    medicines.html
src/main/resources/static/          medicine-catalogue.html
src/main/resources/static/assets/   css/inventory.css, js/inventory.js,
js/medicines.js, js/pharmacy-settings.js
```

# 6. MySQL database tables (owned by Leelavathy)

Five tables, Flyway files `V200__m2_catalogue.sql` and `V210__m2_pharmacy_settings_inventory.sql`. Foreign keys to Divya's `pharmacies` table are created after her migrations ( `V110` ).

**`medicine_categories`** : `id` PK; `name` VARCHAR(80) UNIQUE; `description` VARCHAR(255); `is_active` TINYINT(1).

**`medicines`** : `id` PK; `category_id` FK to `medicine_categories.id` ; `name` VARCHAR(120); `generic_name` VARCHAR(120); `strength` VARCHAR(40); `dosage_form` VARCHAR(20); `manufacturer` VARCHAR(120); `requires_prescription` TINYINT(1); `is_active` TINYINT(1); `created_at` , `updated_at` . UNIQUE ( `name` , `strength` , `dosage_form` ).

**`pharmacy_settings`** : `id` PK; `pharmacy_id` FK to `pharmacies.id` UNIQUE; `contact_phone` VARCHAR(15); `whatsapp_number` VARCHAR(15) (digits with country code, for example `919000000001` ); `delivery_enabled` TINYINT(1) default 0; `delivery_radius_km` DECIMAL(4,1); `delivery_fee` DECIMAL(8,2); `opens_24x7` TINYINT(1); `opening_time` , `closing_time` TIME; `accepts_emergency_orders` TINYINT(1) default 1; `updated_at` .

**`pharmacy_inventory`** : `id` PK; `pharmacy_id` FK to `pharmacies.id` ; `medicine_id` FK to `medicines.id` ; UNIQUE ( `pharmacy_id` , `medicine_id` ); `batch_no` VARCHAR(40); `quantity` INT; `reserved_quantity` INT default 0; `unit_price` DECIMAL(10,2); `low_stock_threshold` INT default 10; `expiry_date` DATE; `data_source` VARCHAR(10) default `SAMPLE` ; `is_active` TINYINT(1); `last_updated_at` DATETIME; `created_at` , `updated_at` . CHECK `quantity >= 0` , `reserved_quantity >= 0` , `reserved_quantity <= quantity` . Indexes: ( `medicine_id` , `pharmacy_id` ), ( `pharmacy_id` , `is_active` ), `expiry_date` .

**`inventory_transactions`** : `id` PK; `inventory_id` FK to `pharmacy_inventory.id` ; `txn_type` VARCHAR(15) ( `STOCK_IN` , `ADJUSTMENT` , `RESERVE` , `RELEASE` , `COMMIT` , `RESTORE` ); `quantity_change` INT (signed); `quantity_after` INT; `reserved_after` INT; `batch_no` VARCHAR(40); `reference_code` VARCHAR(30) NULL (request code from Anusuya's module); `performed_by_user_id` BIGINT NULL (audit only, no FK); `remarks` VARCHAR(255); `created_at` .

**Relationships:** category 1 to many medicines; medicine 1 to many inventory rows; pharmacy 1 to many inventory rows; inventory row 1 to many transactions; pharmacy 1 to 1 settings.

**Stock formula:** `availableQuantity = quantity - reservedQuantity`. A row counts as available only if available is at least the requested quantity, `expiry_date` is after today, and the pharmacy is `APPROVED` and active.

## 7. REST APIs

Base path `/api/v1`, shared response envelope.

| Method | Path | Role | Purpose |
|---|---|---|---|
| GET | `/medicines` | Public | Search by `q`, `categoryId`, `requiresPrescription`, paging |
| GET | `/medicines/{id}` | Public | Medicine details |
| POST | `/medicines` | ADMIN | Create medicine |
| PUT | `/medicines/{id}` | ADMIN | Update medicine |
| PATCH | `/medicines/{id}/status` | ADMIN | Activate or deactivate |
| GET | `/categories` | Public | List categories |
| POST | `/categories` | ADMIN | Create category |
| PUT | `/categories/{id}` | ADMIN | Update category |
| GET | `/inventory` | PHARMACY | Own rows; `q`, `status`, paging |
| GET | `/inventory/{id}` | PHARMACY | One row (own only) |
| POST | `/inventory` | PHARMACY | Add medicine to stock |
| PUT | `/inventory/{id}` | PHARMACY | Edit price, threshold, batch, expiry |
| POST | `/inventory/{id}/stock-in` | PHARMACY | Body `{quantity, batchNo?, expiryDate?}` |
| POST | `/inventory/{id}/adjust` | PHARMACY | Body `{newQuantity, reason}` |
| POST | `/inventory/{id}/confirm` | PHARMACY | Mark row `LIVE`, refresh `last_updated_at` |
| DELETE | `/inventory/{id}` | PHARMACY | Soft remove; blocked if `reserved_quantity > 0` |

| Method | Path | Role | Purpose |
|---|---|---|---|
| GET | `/inventory/{id}/transactions` | PHARMACY | Paged history |
| GET | `/inventory/summary` | PHARMACY | Counts for dashboard cards |
| GET | `/inventory/alerts` | PHARMACY | Low, out and expiring rows |
| GET | `/admin/inventory` | ADMIN | Read-only view by `pharmacyId` |
| GET | `/pharmacy-settings/me` | PHARMACY | Own settings plus read-only address and coordinates |
| PUT | `/pharmacy-settings/me` | PHARMACY | Update settings |

Internal Java services (not REST) for the other modules: `MedicineLookup`, `InventoryQueryService`, `InventoryService`, `PharmacySettingsLookup` (see section 8).

## 8. Integration requirements

### What Leelavathy provides and consumes

- Provides: `MedicineLookup` (to Deepika and Anusuya), `InventoryQueryService` (to Deepika), `InventoryService` (to Anusuya), `PharmacySettingsLookup` (to Deepika and Anusuya), and the public `GET /medicines?q=` used by the search typeahead.
- Consumes from Divya: `CurrentUserProvider` (pharmacy ID, role), `PharmacyLookup` (approved check and demo pharmacy list), `PharmacyApprovedEvent`, the response envelope and the theme. Use temporary stubs until Divya's foundation is merged.
- Consumes from Anusuya (optional, log-only fallback): `NotificationPublisher` for `LOW_STOCK` alerts.
- Contract rules for `InventoryService`: `reserve(pharmacyId, medicineId, qty, refCode)` returns the `inventoryId`; `commit`, `release` and `restore` take `(inventoryId, qty, refCode)`. Each call must be idempotent per inventory row, reference code and transaction type.
- Page links: the catalogue links to `/search.html?medicineId=` (Deepika); settings link to `/profile.html` (Divya) for the location; dashboard cards use `GET /inventory/summary`.

### Shared conventions (identical in all four module documents)

- Base path `/api/v1`. Every response is `{success, message, data, errors[{field,message}], timestamp}`. Lists use `data = {content, page, size, totalElements, totalPages}` with params `page` (from 0), `size` (default 10,

maximum 50) and `sort`.

- Roles: `PATIENT`, `HOSPITAL`, `PHARMACY`, `DELIVERY`, `ADMIN`. The user and pharmacy ID always come from the session through `CurrentUserProvider`, never from the request body.
- Naming: tables plural `snake_case`, entities singular `PascalCase`, JSON `camelCase`, enums `UPPER_SNAKE` stored as strings, `BIGINT` IDs, money `DECIMAL(10,2)` in INR, coordinates `DECIMAL(9,6)`, timestamps in UTC.
- Login is a server-side session; every `POST`, `PUT`, `PATCH`, `DELETE` carries the CSRF header `X-XSRF-TOKEN` (added by `api.js`).
- Stock: `availableQuantity = quantity - reservedQuantity`. Inventory rows are `SAMPLE` or `LIVE`; sample data is never presented as real-time stock.
- Prescription-only medicines need a prescription verified by the pharmacy before a request is accepted.

| HTTP | `errorCode` | When |
|---|---|---|
| 400 | `VALIDATION_ERROR` | Bean validation or bad parameter |
| 401 | `UNAUTHORIZED` | Not logged in or session expired |
| 403 | `FORBIDDEN` | Wrong role or another user's data |
| 403 | `PHARMACY_NOT_APPROVED` | Pharmacy action before approval |
| 404 | `NOT_FOUND` | Missing record |
| 409 | `DUPLICATE_RESOURCE` | Email, licence number or medicine already exists |
| 409 | `INSUFFICIENT_STOCK` | Reservation larger than available stock |
| 409 | `INVALID_STATE_TRANSITION` | Status change not allowed |
| 409 | `NO_DELIVERY_PARTNER` | Delivery not possible; offer self-pickup |
| 422 | `PRESCRIPTION_REQUIRED` | Prescription-only item without a verified prescription |
| 500 | `INTERNAL_ERROR` | Unexpected; generic message only |

### Shared database design (one MySQL database `emergency_med_db`, 19 tables)

| Owner | Module | Tables | Flyway range |
|---|---|---|---|
| Divya | User & Authentication | `users`, `patient_profiles`, `hospital_profiles`, `pharmacies`, `delivery_partner_profiles` | `V100` to `V199` |
| Leelavathy | Medicine & Inventory | `medicine_categories`, `medicines`, `pharmacy_settings`, `pharmacy_inventory`, `inventory_transactions` | `V200` to `V299` |
| Deepika | Medicine Search & Availability | `search_history`, `contact_logs` | `V300` to `V399` |
| Anusuya | Emergency Request & Notification | `emergency_requests`, `request_items`, `prescriptions`, `request_status_history`, `delivery_availability`, `delivery_assignments`, `notifications` | `V400` to `V499` |

Only the owner reads or writes a table directly. Other modules use the Java interfaces below. Anusuya's `request_items` has foreign keys to this module's `medicines` and `pharmacy_inventory`; Deepika's logs reference `medicines`. No `ON DELETE CASCADE` across modules.

| Interface (`com.emergencymed.common.contract`) | Implemented by | Used by | Methods |
|---|---|---|---|
| `CurrentUserProvider` | Divya | all | `Long userId()`, `Role role()`, `Optional<Long> pharmacyId()`, `boolean isAuthenticated()` |
| `PharmacyLookup` | Divya | Leelavathy, Deepika, Anusuya | `findApproved(id)`, `findApprovedWithin(lat, lng, radiusKm)`, `pharmacyIdForUser(userId)` |
| `UserLookup` | Divya | Anusuya | `findById(userId)`, `activeDeliveryPartnerIds()` |
| `MedicineLookup` | Leelavathy | Deepika, Anusuya | `findById(id)`, `searchByName(q, limit)` |

| Interface (`com.emergencymed.common.contract`) | Implemented by | Used by | Methods |
|---|---|---|---|
| `InventoryQueryService` | Leelavathy | Deepika | `findOffers(medicineId, qty, pharmacyIds)`, `findAlternatives(medicineId, pharmacyIds)` |
| `InventoryService` | Leelavathy | Anusuya | `reserve`, `commit`, `release`, `restore` |
| `PharmacySettingsLookup` | Leelavathy | Deepika, Anusuya | `find(pharmacyId)` |
| `DeliveryAvailabilityQuery` | Anusuya | Deepika | `statusFor(pharmacyId)`: `AVAILABLE`, `BUSY`, `UNAVAILABLE` |
| `NotificationPublisher` | Anusuya | Divya, Leelavathy, Anusuya | `publish(userId, type, title, message, requestId)` |

Shared DTO shapes returned by her interfaces: `MedicineView(id, name, genericName, strength, form, categoryName, requiresPrescription, isActive)`, `InventoryOffer(inventoryId, pharmacyId, medicineId, availableQuantity, unitPrice, batchNo, expiryDate, stockStatus, dataSource, lastUpdatedAt)`, `PharmacySettingsView(pharmacyId, contactPhone, whatsappNumber, deliveryEnabled, deliveryRadiusKm, deliveryFee, opens24x7, openingTime, closingTime)`.

## 9. Validation, exception handling and security

| Field | Rule |
|---|---|
| Medicine name, strength | Required; duplicate name plus strength plus form returns 409 |
| Quantity | Integer 0 to 100000 |
| Unit price | Greater than 0 and at most 100000, two decimals |
| Batch number | 3 to 40 characters: letters, digits, hyphen |
| Expiry date | After today on add and stock-in; an expired row is `EXPIRED` and never offered |
| Low-stock threshold | 0 to 1000 |
| WhatsApp number | 11 to 13 digits including country code, no `+` |

| Field | Rule |
|---|---|
| Delivery radius and fee | 1 to 50 km; fee 0 to 500 |
| Opening hours | Closing after opening unless `opens_24x7` |
| Adjust reason | Required, at least 5 characters |

- 409 `INSUFFICIENT_STOCK` when a reserve exceeds available stock; 404 `NOT_FOUND` when a row belongs to another pharmacy (never reveal it exists).
- Only `APPROVED` and active pharmacies can change inventory, otherwise 403 `PHARMACY_NOT_APPROVED`.
- Catalogue writes need `ROLE_ADMIN`; inventory and settings writes need `ROLE_PHARMACY`.
- Every stock change is atomic and writes a transaction row in the same database transaction. `reserve`, `commit`, `release` and `restore` are idempotent per inventory row, reference code and transaction type.
- Pessimistic row locks and database `CHECK` constraints prevent negative stock even with parallel requests.

## 10. Testing scenarios and expected output

| Scenario | Steps | Expected |
|---|---|---|
| Add stock | Add Paracetamol 650 mg, quantity 50, price 28.50 | 201; row `IN_STOCK`, `LIVE`; one `STOCK_IN` transaction |
| Duplicate row | Add the same medicine again | 409 `DUPLICATE_RESOURCE` with a hint to use Stock in |
| Past expiry | Add with yesterday as expiry | 400 on `expiryDate` |
| Low stock | Adjust a row to 8 with threshold 10 | Amber badge; `LOW_STOCK` notification sent |
| Out of stock | Adjust to 0 | Red badge; not returned by `findOffers` |
| Reserve | `reserve(qty 5)` on available 20 | `reserved_quantity` 5, available 15, `RESERVE` transaction |
| Over-reserve | Two requests of 15 each on available 20 | First succeeds; second 409 `INSUFFICIENT_STOCK`; stock unchanged |

| Scenario | Steps | Expected |
|---|---|---|
| Commit then cancel | Reserve 5, commit 5, restore 5 | Quantity returns to the original; three transactions in order |
| Idempotency | Send the same `commit` (same reference code) twice | Stock changes once |
| Other pharmacy | Pharmacy A edits a row of pharmacy B | 404 `NOT_FOUND` |
| Sample to live | Press Confirm on a sample row | Badge changes from Demo data to Live; `last_updated_at` is now |
| Concurrency | Two parallel reserves of the last unit | Exactly one succeeds |
| Approval hook | Divya approves a pharmacy | Default `pharmacy_settings` row exists |